

```

import glob
import pickle
import numpy as np

from music21 import converter, instrument, note, chord, stream

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, LSTM
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import ModelCheckpoint

# -----
# STEP 1: LOAD MIDI FILES
# -----

def get_notes():
    notes = []

    for file in glob.glob("dataset/*.mid"):
        print("Processing:", file)

        midi = converter.parse(file)

        try:
            parts = instrument.partitionByInstrument(midi)
            notes_to_parse = parts.parts[0].recurse()
        except:
            notes_to_parse = midi.flat.notes

        for element in notes_to_parse:

            if isinstance(element, note.Note):
                notes.append(str(element.pitch))

            elif isinstance(element, chord.Chord):
                notes.append('.'.join(str(n) for n in element.normalOrder))

    return notes

# -----
# STEP 2: PREPARE TRAINING DATA
# -----

def prepare_sequences(notes, sequence_length=100):

    pitchnames = sorted(set(notes))

    note_to_int = dict(
        (note, number)
        for number, note in enumerate(pitchnames)
    )

    network_input = []
    network_output = []

    for i in range(len(notes) - sequence_length):

        sequence_in = notes[i:i + sequence_length]
        sequence_out = notes[i + sequence_length]

        network_input.append(
            [note_to_int[char] for char in sequence_in]
        )

        network_output.append(

```

```

        note_to_int[sequence_out]
    )

    n_patterns = len(network_input)

    network_input = np.reshape(
        network_input,
        (n_patterns, sequence_length, 1)
    )

    network_input = network_input / float(len(pitchnames))

    network_output = to_categorical(network_output)

    return network_input, network_output, pitchnames

```

```

# -----
# STEP 3: BUILD LSTM MODEL
# -----

```

```

def create_network(network_input, n_vocab):

    model = Sequential()

    model.add(
        LSTM(
            512,
            input_shape=(
                network_input.shape[1],
                network_input.shape[2]
            ),
            return_sequences=True
        )
    )

    model.add(Dropout(0.3))

    model.add(
        LSTM(
            512,
            return_sequences=True
        )
    )

    model.add(Dropout(0.3))

    model.add(LSTM(512))

    model.add(Dense(256, activation='relu'))

    model.add(Dropout(0.3))

    model.add(Dense(n_vocab, activation='softmax'))

    model.compile(
        loss='categorical_crossentropy',
        optimizer='adam'
    )

    return model

```

```

# -----
# STEP 4: TRAIN MODEL
# -----

```

```

def train_model(model, network_input, network_output):

    checkpoint = ModelCheckpoint(
        "best_model.keras",
        monitor='loss',
        verbose=1,
        save_best_only=True
    )

    model.fit(
        network_input,
        network_output,
        epochs=50,
        batch_size=64,
        callbacks=[checkpoint]
    )

# -----
# STEP 5: GENERATE MUSIC
# -----

def generate_notes(model, network_input, pitchnames, num_notes=500):

    int_to_note = dict(
        (number, note)
        for number, note in enumerate(pitchnames)
    )

    start = np.random.randint(
        0,
        len(network_input) - 1
    )

    pattern = network_input[start]
    prediction_output = []

    for _ in range(num_notes):

        prediction_input = np.reshape(
            pattern,
            (1, len(pattern), 1)
        )

        prediction = model.predict(
            prediction_input,
            verbose=0
        )

        index = np.argmax(prediction)

        result = int_to_note[index]

        prediction_output.append(result)

        pattern = np.append(
            pattern,
            [[index / float(len(pitchnames))]],
            axis=0
        )

        pattern = pattern[1:]

    return prediction_output

```

```

# -----
# STEP 6: CREATE MIDI FILE
# -----

def create_midi(prediction_output):

    offset = 0
    output_notes = []

    for pattern in prediction_output:

        if '.' in pattern or pattern.isdigit():

            notes_in_chord = pattern.split('.')
            chord_notes = []

            for current_note in notes_in_chord:

                new_note = note.Note(int(current_note))
                new_note.storedInstrument = instrument.Piano()
                chord_notes.append(new_note)

            new_chord = chord.Chord(chord_notes)
            new_chord.offset = offset
            output_notes.append(new_chord)

        else:

            new_note = note.Note(pattern)
            new_note.offset = offset
            new_note.storedInstrument = instrument.Piano()

            output_notes.append(new_note)

    offset += 0.5

    midi_stream = stream.Stream(output_notes)

    midi_stream.write(
        'midi',
        fp='generated_music.mid'
    )

    print("Generated MIDI saved as generated_music.mid")

# -----
# MAIN FUNCTION
# -----

def main():

    print("Loading MIDI dataset...")
    notes = get_notes()

    print("Preparing sequences...")
    network_input, network_output, pitchnames = prepare_sequences(notes)

    print("Building model...")
    model = create_network(
        network_input,
        len(pitchnames)
    )

    print("Training model...")

```

```
train_model(  
    model,  
    network_input,  
    network_output  
)  
  
print("Generating music...")  
generated_notes = generate_notes(  
    model,  
    network_input,  
    pitchnames  
)  
  
create_midi(generated_notes)  
  
print("Done!")  
  
if __name__ == "__main__":  
    main()
```